Relatório Técnico - DR1 TP3

1. Identificação

- Nome completo: Renato Noronha Hack
- Disciplina: Fundamentos de Verilog e FPGA
- Professor: Vitor Amadeu
- Data: 14/03/2026

2. Etapa 1 - Função booleana e fundamentos

Item 1 - Interpretação da função

Função fornecida:

- F(A,B,C) = Sm(1,3,5,7)

Combinações de entrada que tornam F = 1:

- 001, 011, 101, 111

Interpretação lógica:

- A saída F vale 1 em todos os casos em que C = 1.
- As variáveis A e B não alteram o resultado nesses casos.
- Portanto, a função depende apenas de C.

3. Etapa 1 - Tabela verdade (Item 2)

Decimal	A	B	C	F	Produto Fundamental
0	0	0	0	0	A'B'C'
1	0	0	1	1	A'B'C
2	0	1	0	0	A'BC'
3	0	1	1	1	A'BC
4	1	0	0	0	AB'C'
5	1	0	1	1	AB'C
6	1	1	0	0	ABC'
7	1	1	1	1	ABC

Consistência da tabela com a definição do enunciado:

- A tabela está consistente com Sm(1,3,5,7), pois F = 1 exatamente nesses mintermos.

4. Etapa 2 - Formas canônicas

Item 3 - Forma SOP (Soma de Produtos)

Expressão SOP canônica:

```
F = A'B'C + A'BC + AB'C + ABC
```

Item 4 - Forma POS (Produto de Somas)

Expressão POS canônica:

```
F = (A + B + C)(A + B' + C)(A' + B + C)(A' + B' + C)
```

5. Etapa 3 - Mapa de Karnaugh e minimização

Item 5 - Mapa de Karnaugh

AB \ C	C'	C
A'B'	0	1
A'B	0	1
AB	0	1
AB'	0	1

Agrupamentos identificados:

- Grupo 1: coluna inteira C, com 4 células de valor 1.
- Grupo 2: não necessário.

Item 6 - Expressão minimizada

Expressão minimizada:

```
F = C
```

Comparação de complexidade:

- Termos na forma canônica SOP: 4 termos de 3 variáveis.
- Termos na forma minimizada: 1 termo de 1 variável.
- Conclusão: houve redução significativa da complexidade lógica.

6. Etapa 4 - Implementação combinacional em Verilog

Item 7 - Arquivo `comb_logic.v`

```
module Comb_Logic (
    input  wire A,
    input  wire B,
    input  wire C,
    output wire F
);
    assign F = C;
endmodule
```

Item 8 - Testbench e resultados (entradas -> saída observada)

Tabela de observação da simulação:

A	B	C	F observado
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Conclusão da verificação:

- A simulação confirma F = C para todas as 8 combinações de entrada.

7. Etapa 5 - Mux2to1

O arquivo de referência citado no relatório é `src/mux2to1.v`. Seu conteúdo é o seguinte:

```
`timescale 1ns/1ps

// TP3 - Etapa 5
// Multiplexador 2:1 combinacional.
module Mux2to1 (
    input  wire D0,
    input  wire D1,
    input  wire S,
    output wire Y
);
    assign Y = S ? D1 : D0;

endmodule
```

8. Etapa 6 - Reg1 e Counter2bit

Item 10 - Reg1

Papel do flip-flop no armazenamento de estado:

- O flip-flop D é o elemento sequencial responsável por armazenar 1 bit de informação.
- A cada borda de subida do clock, ele copia o valor presente em `D` para a saída `Q`.
- Dessa forma, o circuito passa a ter memória, pois o valor de `Q` permanece armazenado até a próxima borda de clock.
- No módulo `Reg1`, o reset síncrono ativo em nível alto permite forçar `Q = 0` em uma borda de subida, reinicializando o estado do circuito de forma controlada.

Conteúdo do arquivo `src/reg1.v`:

```
`timescale 1ns/1ps
```

```
// TP3 - Etapa 6
// Flip-flop D com reset sincrono ativo em nivel alto.
module Reg1 (
    input  wire D,
    input  wire clk,
    input  wire reset,
    output reg  Q = 1'b0
);
    always @(posedge clk) begin
        if (reset)
            Q <= 1'b0;
        else
            Q <= D;
    end

endmodule
```

Item 11 - Counter2bit

Sequência observada do contador:

- 00 -> 01 -> 10 -> 11 -> 00

Conteúdo do arquivo `src/counter2bit.v`:

```
`timescale 1ns/1ps

// TP3 - Etapa 6
// Contador binario de 2 bits usando dois flip-flops Reg1.
module Counter2bit (
    input  wire       clk,
    input  wire       reset,
    output wire [1:0] Q
);
    wire d0;
    wire d1;

    // Proximo estado para contador modulo 4:
    // 00 -> 01 -> 10 -> 11 -> 00
    assign d0 = ~Q[0];
    assign d1 = Q[1] ^ Q[0];

    Reg1 ff0 (
        .D(d0),
        .clk(clk),
        .reset(reset),
        .Q(Q[0])
    );
```

```
    Reg1 ff1 (
        .D(d1),
        .clk(clk),
        .reset(reset),
        .Q(Q[1])
    );

endmodule
```

9. Etapa 7 - Integração no top e validação física

Item 12

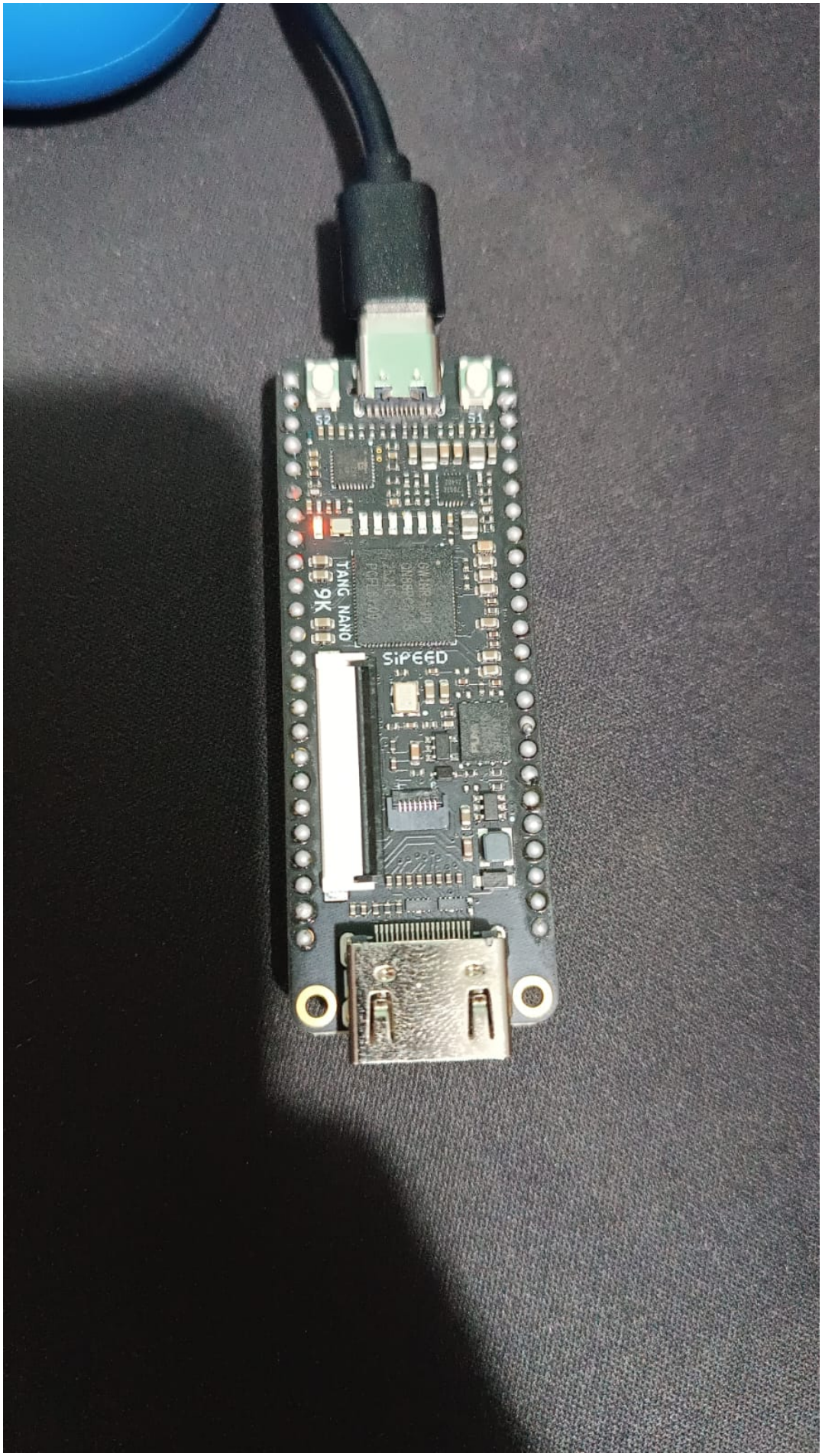
- Mapeamento de pinos: clk = 52, btn_inc_n = 3, led0 = 10, led1 = 11.
- Montagem física: uso apenas da Tang Nano 9K, sem protoboard, com botão e LEDs onboard.
- Resultado na placa Tang Nano 9K: o contador iniciou em 00 e incrementou para 01, 10, 11 e 00 a cada apertar e soltar do botão onboard.

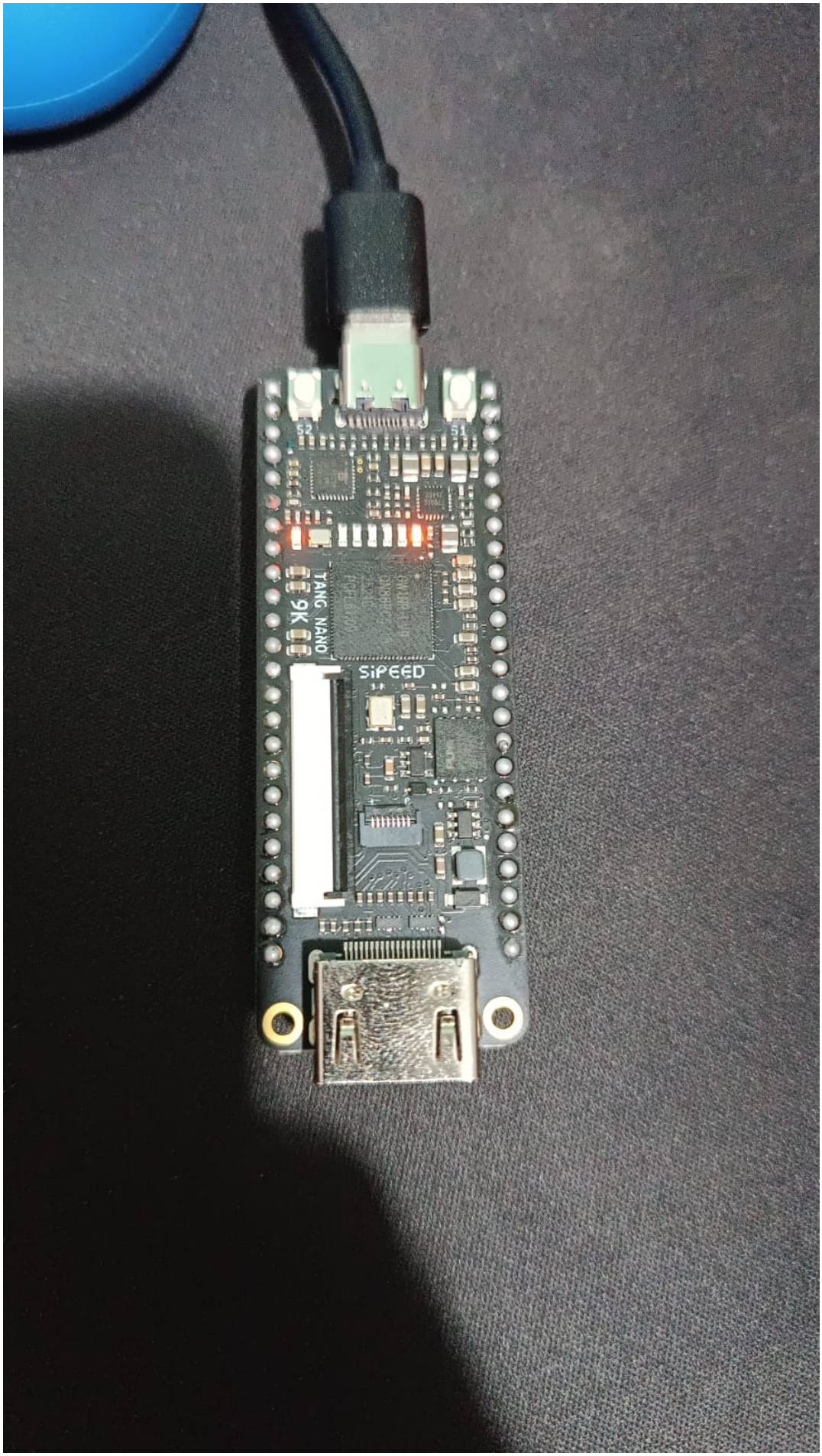
Conteúdo do arquivo `src/top.v`:

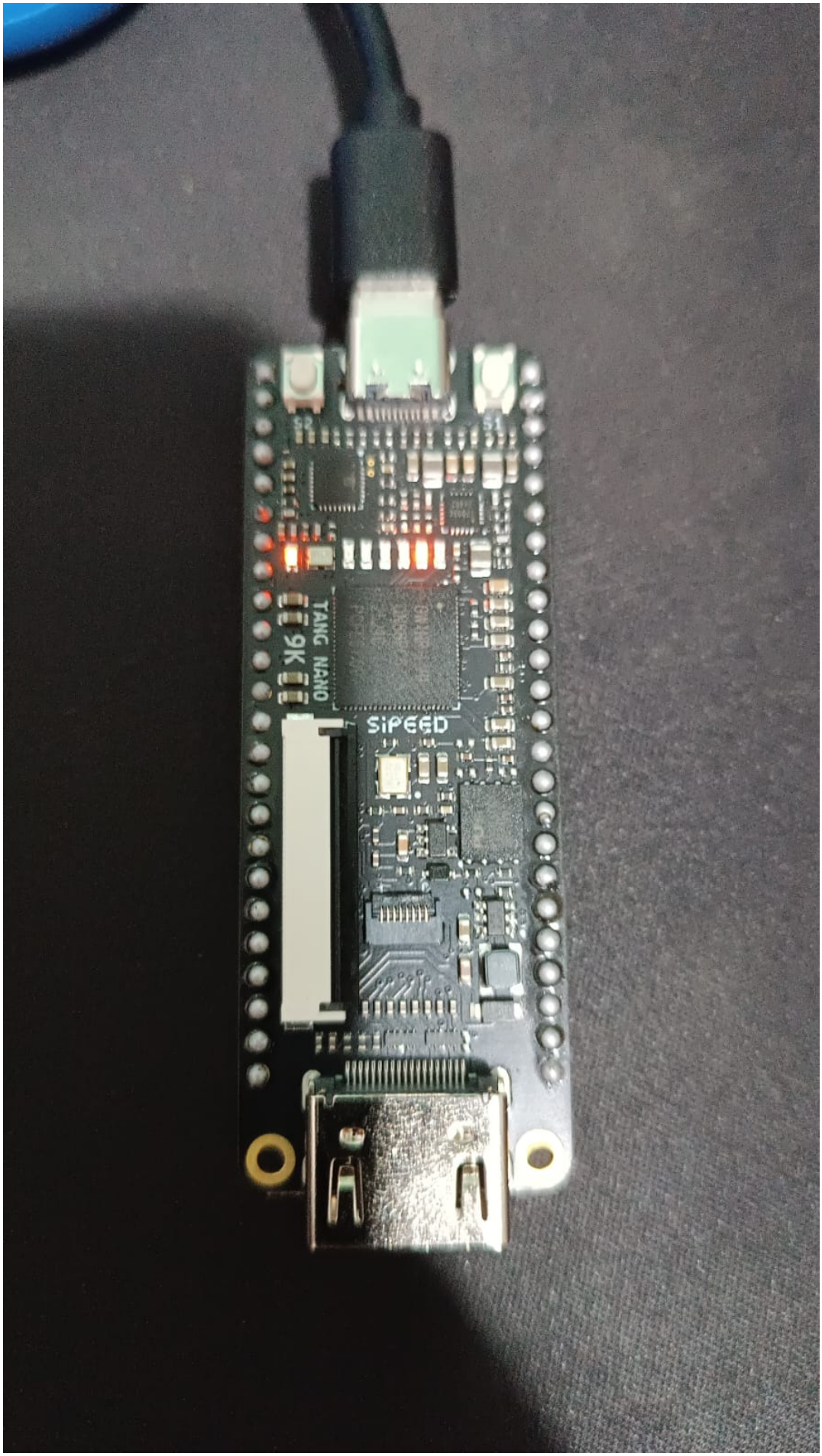
```
`timescale 1ns/1ps

// TP3 - Etapa 7
// Top-level para validar fisicamente o contador com:
// - clock onboard de 27 MHz
// - botao onboard para incrementar ao soltar
// - dois LEDs onboard, ativos em nivel baixo
module top (
    input  wire clk,
    input  wire btn_inc_n,
    output wire led0,
    output wire led1
);
    localparam integer DEBOUNCE_CLKS = 270_000;

    reg [18:0] debounce_count = 19'd0;
    reg        btn_sync_0     = 1'b1;
    reg        btn_sync_1     = 1'b1;
    reg        btn_stable     = 1'b1;
    reg        inc_pulse      = 1'b0;
    wire [1:0] count;

    // Sincroniza e faz debounce do botao onboard.
    // O pulso de incremento acontece no momento em que o botao e solto:
    // pressionado = 0, solto = 1.
    always @(posedge clk) begin
        btn_sync_0 <= btn_inc_n;
        btn_sync_1 <= btn_sync_0;
```

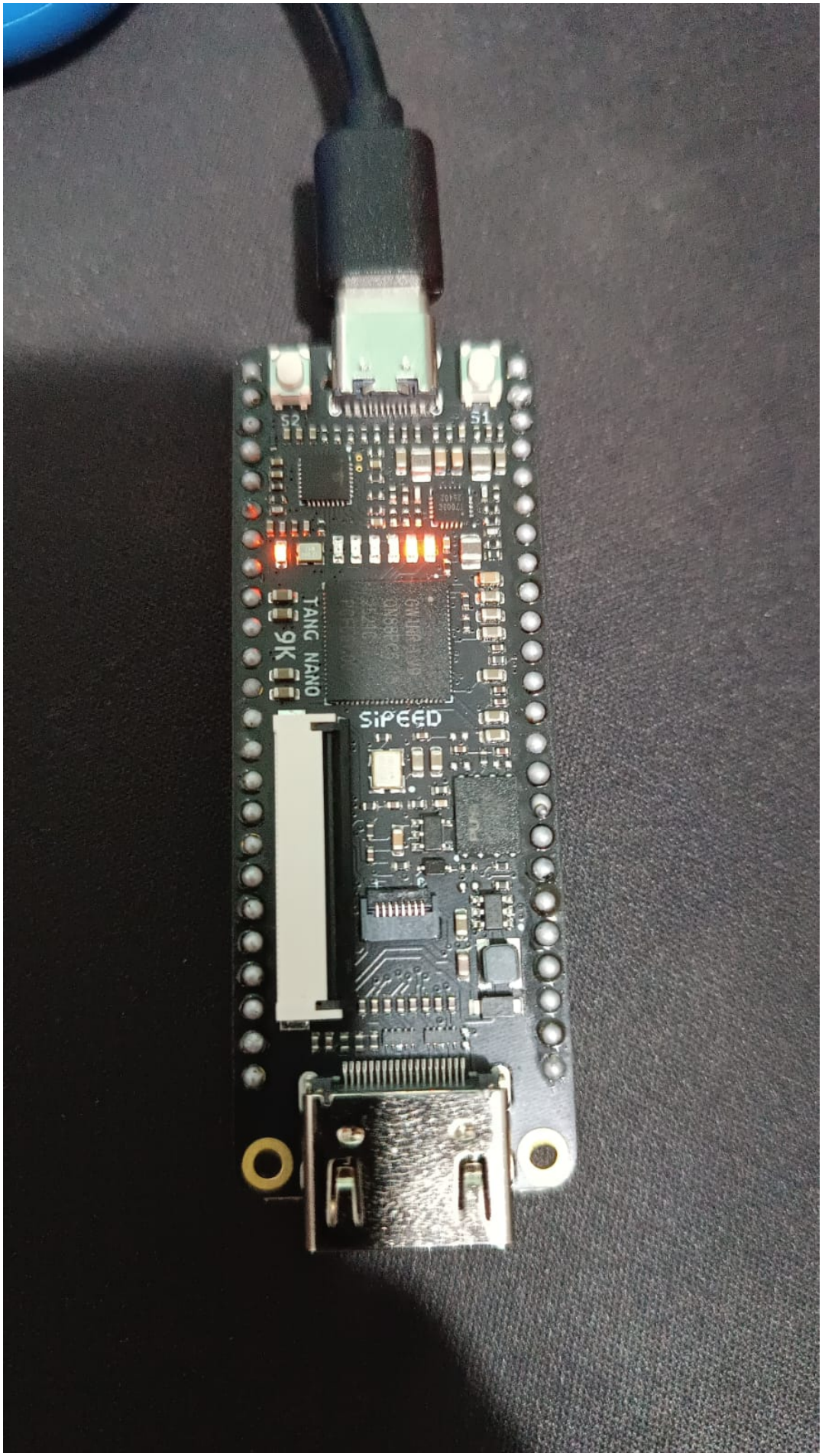
```
        inc_pulse  <= 1'b0;

        if (btn_sync_1 == btn_stable) begin
            debounce_count <= 19'd0;
        end else if (debounce_count == DEBOUNCE_CLKS - 1) begin
            if ((btn_stable == 1'b0) && (btn_sync_1 == 1'b1))
                inc_pulse <= 1'b1;

            btn_stable     <= btn_sync_1;
            debounce_count <= 19'd0;
        end else begin
            debounce_count <= debounce_count + 1'b1;
        end
    end

    Counter2bit u_counter (
        .clk(inc_pulse),
        .reset(1'b0),
        .Q(count)
    );

    // LEDs onboard da Tang Nano 9K sao ativos em nivel baixo.
    assign led0 = ~count[0];
    assign led1 = ~count[1];

endmodule
```

Evidências fotográficas:


S2
S1
TANG NANO
9K
SIPEED


S2
S1
TANG NANO
9K
SIPEED


TANG NANO
9K
SIPEED


S2
S1
TANG NANO
9K
SiPEED

10. Conclusão

- O desenvolvimento deste trabalho permitiu revisar todo o fluxo básico de projeto em FPGA, partindo da definição de uma função booleana, passando pela minimização com mapa de Karnaugh e chegando à implementação em Verilog.
- Na parte combinacional, foi confirmado por simulação que a expressão minimizada da função proposta é simplesmente `F = C`.
- Na parte sequencial, o uso do flip-flop D mostrou como o FPGA pode armazenar estado e, a partir disso, implementar um contador binário de 2 bits.
- A validação final na Tang Nano 9K confirmou o funcionamento prático do sistema, com o contador avançando corretamente e os LEDs representando visualmente os estados 00, 01, 10 e 11.

11. Anexos

- Capturas de simulação.
- Captura do mapa de Karnaugh.
- Fotos da placa em funcionamento:
  - `00.jpeg`
  - `01.jpeg`
  - `10.jpeg`
  - `11.jpeg`